

dgrid_dshot_rx — DShot receive + bidirectional telemetry reply

Block spec for review · DG32 · 13 Sep 2026 · draft 1 Author: DG32 engineering (drafted with Claude). Reviewer: Ayaz Khan.

0. Ayaz's questions — answered from the RTL

#	Question	Answer	Source
1	Do the timers at 0xD4/0xD5 have pin-routed input capture?	No. dgrid_timer's entire non-AXI interface is tick_o and irq_o. No input pin, no capture register. DShot300-via-timer is not an interim path.	rtl/dgrid_timer.sv module ports
1b	Is there <i>any</i> edge-timestamp hardware on the die?	Yes — on the encoder. dgrid_encoder (0xDA) has a free-running 32-bit timestamp_q captured on Hall/quadrature edges ("Hall decode with edge timestamps"). It is functionally input-capture, bonded to the encoder/Hall pads, not the timers.	rtl/dgrid_encoder.sv:26,91-95
2	Datasheet says 14 GPIO, pinout bonds 3 — which is right?	The pinout. NGPIO = 3. It was 14; 11 pads were reclaimed for I ² C (2) and scan/DFT (4). The datasheet is stale and must be corrected.	rtl/dg32_lite_top.sv:149
3	28 KB vs 32 KB SRAM	Both positions are legitimate and they answer different questions — see §9. The codex branch is already running both a 24 KB variant (2dom/12) and a keep-32 KB spread-floorplan test (2dom/13). Those runs decide it with data.	autoresearch/configs/2dom/12,13
4	"DShot bidirectional / RPM telemetry — settled: not supported"	Re-opened by this spec. Ayaz's case is correct: it cannot be done in firmware, and it is required for DG32-as-ESC.	this document

1. Why firmware cannot do this (confirming Ayaz's argument)

DShot600 is 1.67 $\mu\text{s}/\text{bit}$; the 0/1 decision is a high-time of 625 ns vs 1250 ns. At 50 MHz that is 31 vs 62 cycles, so the decode needs $\sim\pm 15$ -cycle (± 300 ns) edge placement — achievable only by a busy-wait loop with interrupts off for the full 16-bit frame ($\sim 1,340$ cycles at DShot600, $\sim 2,700$ at DShot300). That collides with:

- the **5 μs FOC loop** (250 cycles) — a single DShot frame is 5 $\times$ longer than the entire control period;

- the **lockstep compare window** — `dgrid_lockstep_check` compares every committed store; a firmware bit-bang that branches on pad reads produces store streams whose timing depends on an asynchronous input, which is exactly the class of behaviour the checker is designed to flag.

And **bidirectional reply is impossible in firmware regardless of speed**: the ESC must turn the pad from input to output $\sim 30\ \mu\text{s}$ after the command frame ends and drive a 21-bit reply — the pad has no output-enable today. This is a pad-ring fact, not a software one.

Conclusion: hardware RX + hardware reply, or no ESC role.

2. What exists today (baseline)

`dgrid_dshot` (slot 0xC, `rtl/dgrid_dshot.sv`) is **transmit-only**: 4 channels, one shared bit-time counter, per-channel 16-bit frame + CRC4, programmable `BITPERIOD` / `T0H` / `T1H`. Its only pins are `output [3:0] dshot_o`. The four channels reach the pads only through `SYSCTL PINMUX[0]`, which AND-ORs them onto the PWM high-side pads `AH` / `BH` / `CH` / `SAMPLE_TRIG` (`dg32_lite_top.sv:775-792`). The low-side PWM pads are never muxed, so a mis-set mux cannot shoot through a bridge.

Pad budget is 44/44 with zero spare. There is no pad for a receiver. The RX path therefore **must** reuse the same four pads, which means those pads become bidirectional. That is the single most consequential change in this spec (§7).

3. Protocol facts the design depends on

3.1 Command frame (FC → ESC), what RX decodes

16 bits, MSB first: `[15:5] throttle (11 b)` · `[4] telemetry request` · `[3:0] CRC4`. Each bit is one pulse of fixed period; **high-time encodes the value** (long = 1, short = 0).

speed	bit period	T0H	T1H	@50 MHz period / T0H / T1H (cycles)
DShot150	6.67 μs	2.50 μs	5.00 μs	333 / 125 / 250
DShot300	3.33 μs	1.25 μs	2.50 μs	167 / 62 / 125
DShot600	1.67 μs	0.625 μs	1.25 μs	83 / 31 / 62
DShot1200	0.83 μs	0.312 μs	0.625 μs	42 / 16 / 31

CRC4, normal mode: `crc = n0 ^ n1 ^ n2` of the 12-bit `{throttle, telem}` — identical to the TX engine's `crc4_of()`. **CRC4, bidirectional mode:** `crc = ~(n0 ^ n1 ^ n2) & 0xF`. The inverted CRC is how the protocol marks a bidirectional-capable frame.

3.2 Bidirectional mode signalling

In bidirectional DShot the **line is inverted**: idle **high**, bits are **low** pulses (high-time above becomes low-time). The ESC recognises bidirectional mode from the inverted idle level and inverted CRC. RX needs an `INVERT` control bit that flips the input sense before the width measurement.

3.3 Telemetry reply (ESC → FC), what the reply engine sends

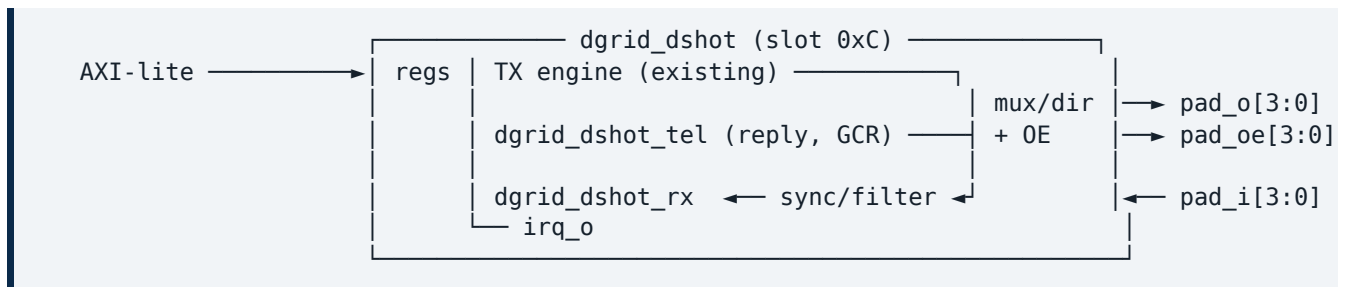
The reply is **not a DShot command frame**, so it cannot reuse the TX engine's shifter unmodified:

1. Payload: 16 bits = [15:4] eRPM period (3-bit exponent e , 9-bit mantissa m ; $\text{period}_{\mu\text{s}} = m \ll e$) · [3:0] CRC where $\text{crc} = \sim(v \wedge v \gg 4 \wedge v \gg 8) \& 0xF$ over the 12-bit period field.
2. **GCR 4b→5b** encode each nibble (table: 19 1B 12 13 1D 15 16 17 1A 09 0A 0B 1E 0D 0E 0F) → 20 bits.
3. **Transition-encode** to 21 bits: $\text{out}[0] = 0$; $\text{out}[i] = \text{out}[i-1] \wedge G[i-1]$ — a 1 in the GCR stream is a level *transition*, not a level.
4. Transmit **inverted**, at **5/4 the command bit rate** (DShot600 → 750 kbit/s, $1.33 \mu\text{s/bit} = 66.7$ cycles), starting **~30 μs** after the command frame's last edge.

The FC decodes edges back with $G = s \wedge (s \gg 1)$. This is the Betaflight/Bluejay/AM32 convention; get any of the three steps wrong and every FC on the market reports CRC errors.

4. Architecture

Recommendation: extend slot 0xC in place, not a new slot. The RX and reply engines are added *inside* `dgrid_dshot` as two self-contained sub-modules (`dgrid_dshot_rx`, `dgrid_dshot_tel`), sharing the existing AXI slave, the pad mux and the bit-timing registers. Reasons: (a) the address map has no free slot without touching the decoder and error-slave logic that is frozen for tape-in; (b) RX and TX are mutually exclusive on the same pads, so one owner of the pad direction is safer than two; (c) `BITPERIOD/T0H/T1H` already encode the speed class — RX derives its threshold from them by default.



Per channel (×4): RX decoder, reply shifter, direction FSM. Shared: bit-time counter (existing), threshold/timeout/turnaround registers, 2-FF synchronisers.

4.1 RX decoder (per channel)

- Input: `pad_i` → 2-FF synchroniser → 2-sample glitch filter → `INVERT XOR` → `rx_level`.
- Rising edge of `rx_level` starts a 12-bit high-time counter; falling edge stops it and compares to `RXTHRESH`: $\text{bit} = (\text{high_time} > \text{RXTHRESH})$.
- 16-bit shift register, MSB first. A 4-bit bit counter completes the frame at 16.
- **Frame boundary**: an idle period longer than `RXTIMEOUT` cycles (default $\approx 2 \times \text{BITPERIOD}$) resets the bit counter — this is how a mid-frame glitch or a lost edge cannot desynchronise the receiver permanently.
- On 16 bits: compute CRC4 per §3.1 (mode-dependent), latch `{throttle, telem}`, set `VALID`; set `CRC_ERR` instead if mismatch; set `OVERRUN` if `VALID` was still set (firmware hadn't read the previous frame).

- Synchroniser + filter add a fixed 3-cycle (60 ns) latency to *both* edges, so measured width is unaffected.

4.2 Reply engine `dgrid_dshot_tel` (per channel)

- Armed when `TELEM_EN=1` and a valid frame with `telem=1` arrives (or `TELEM_ALWAYS=1` — bidirectional FCs expect a reply on every frame).
- Waits `TURNAROUND` cycles from the command frame's last edge, then asserts `pad_oe`, drives the 21-bit stream from `TELEMn` at `TELEMPERIOD` cycles/bit (inverted), then releases `pad_oe` and returns the pad to input.
- GCR + transition encoding (§3.3) done in hardware from the 12-bit period value firmware writes to `TELEMn`. Firmware supplies only `{e, m}`; CRC, GCR and transition coding are hardware. This keeps the firmware side to one register write per commutation-period update.
- If a new command frame's first edge arrives while a reply is being driven, the reply is **aborted and OE released within one cycle** — never fight the FC for the wire. `TEL_ABORT` status bit is set.

4.3 Direction ownership

`pad_oe` per channel is the OR of: TX engine `BUSY` (existing TX role, always drives) and `dgrid_dshot_tel` driving. In RX-only or RX+reply roles the TX engine is never started, so the pad idles as input. `PINMUX[0]` continues to select DShot vs PWM; **PWM-role pads remain output-only** (OE forced high) so the bridge-safety property is preserved.

5. Register map (slot `0xC`, `addr[7:0]`; existing registers `0x00-0x20` unchanged)

addr	name	bits	R/W	description
0x24	<code>RXCTRL</code>	[0] <code>RX_EN</code>	RW	enable receivers (all 4 channels)
		[1] <code>INVERT</code>	RW	bidirectional-mode sense: idle high, low pulses, inverted CRC
		[2] <code>TELEM_EN</code>	RW	reply engine armed on frames with telem bit set
		[3] <code>TELEM_ALWAYS</code>	RW	reply on every valid frame regardless of telem bit
		[4] <code>IRQ_FRAME_EN</code>	RW	interrupt on any channel <code>VALID</code>
		[5] <code>IRQ_ERR_EN</code>	RW	interrupt on any channel <code>CRC_ERR</code> / <code>OVERRUN</code> / <code>TEL_ABORT</code>
		[7:6]	—	reserved, read 0
0x28	<code>RXTHRESH</code>	[11:0]	RW	high-time (cycles) above which a bit reads 1. Default $(T0H + T1H) / 2$ if written 0.
0x2C	<code>RXTIMEOUT</code>	[11:0]	RW	idle cycles that terminate/reset a partial frame. Default $2 \times BITPERIOD$ if 0.

addr	name	bits	R/W	description
0x30	RXFRAME0	[10:0] THROTTLE	RO	received throttle, channel 0
		[11] TELEM	RO	telemetry-request bit as received
		[12] VALID	RO/W1C	frame received, CRC good. Cleared by writing 1 (see note on why not read-to-clear)
		[13] CRC_ERR	RO/W1C	frame received, CRC bad
		[14] OVERRUN	RO/W1C	a new frame completed before VALID was cleared
		[15] TEL_ABORT	RO/W1C	reply aborted by an incoming command edge
0x34	RXFRAME1	as above		channel 1
0x38	RXFRAME2	as above		channel 2
0x3C	RXFRAME3	as above		channel 3
0x40	TURNAROUND	[15:0]	RW	cycles from command frame's last edge to reply start. DShot600 nominal 30 μ s = 1500 .
0x44	TELEMPERIOD	[11:0]	RW	reply bit period in cycles. Default $\text{BITPERIOD} \times 4 / 5$ if 0 (DSHOT600 $\rightarrow$ 67).
0x48	TELEM0	[11:0] {e[2:0], m[8:0]}	RW	eRPM period to reply on channel 0. Hardware appends CRC, GCR-encodes, transition-encodes.
0x4C	TELEM1		RW	channel 1
0x50	TELEM2		RW	channel 2
0x54	TELEM3		RW	channel 3
0x58	RXSTATUS	[3:0] VALID[ch] · [7:4] CRC_ERR[ch] · [11:8] OVERRUN[ch] · [15:12] TEL_BUSY[ch] · [19:16] PAD_DIR[ch] (1 = driving)	RO	one-read snapshot of all four channels
0x5C	RXIRQ	[0] FRAME · [1] ERR	RO/W1C	interrupt pending flags

Reset state: RX_EN=0, all thresholds 0 ($\rightarrow$ derived defaults), all pads input-disabled *only* if the wrapper's pad config leaves them so — see §7.

Note on clearing VALID: W1C is the safe default and matches the other DG32 peripherals. Read-to-clear is *not* recommended: the lockstep checker mirrors bus reads to the CHECKER core two cycles late, so a read with a side-effect would be observed by both cores but the side-effect happens once — safe in practice, but it is exactly the kind of subtlety the design avoids elsewhere. Keep W1C.

6. Interrupts

One new source into `dgrid_irq` (0xDF): `dshot_rx_irq = (IRQ_FRAME_EN & |VALID) | (IRQ_ERR_EN & | (CRC_ERR|OVERRUN|TEL_ABORT))`. Level, cleared by W1C of the underlying flags. `dgrid_irq` has a 16-bit raw vector with bits 0–6 and 8 (attention, 2dom) used; **bit 7 is 1'b0 today and is the natural slot** (`rtl/dgrid_irq.sv:58`). Bits 9–15 are also free. Follow the existing convention: add `dshot_rx_i = 1'b0` as a defaulted port so the frozen lite netlist is unchanged when the port is left unconnected — and tie it explicitly in the top (`sv2v` drops port defaults).

7. Pad-ring change — the real cost of this feature

The four DShot pads are **io[26] P_PWM_AH, io[28] P_PWM_BH, io[30] P_PWM_CH, io[32] P_PWM_TRIG** (`openframe/openframe_project_wrapper.v:134-136`). Today they are class `PC_OUT` (`pad_class()` at line 470: `i >= P_PWM_AH && i <= P_PWM_TRIG`), i.e. `gpio_dm = 110`, `gpio_inp_dis = 1`, `oeb` tied to constant drive (lines 408–414).

The wrapper **already has the class we need**: `PC_BIDIR` (`gpio_dm = 110`, `gpio_inp_dis = 0`, `oeb` dynamic) is what QSPI IO0–3, I²C and the three GPIOs use. So the change is three edits, not a new pad mode:

edit	today	required
<code>pad_class()</code>	<code>io[26..32] → PC_OUT</code>	<code>io[26], io[28], io[30], io[32] → PC_BIDIR</code> ; <code>io[27], io[29], io[31]</code> stay <code>PC_OUT</code>
<code>oeb_v[P_PWM_AH/BH/CH/TRIG]</code>	<code>gpio_loopback_zero</code> (always drive)	<code>~pad_oe[ch]</code> from <code>dgrid_dshot</code> , through <code>u_glue</code> like <code>gpio3_oeb</code> (<code>oeb</code> is active-low)
<code>gpio_in[P_PWM_AH/BH/CH/TRIG]</code>	unused	<code>→ dgrid_dshot.pad_i[3:0]</code> through the top-level

`gpio_ib_mode_sel`, `gpio_vtrip_sel`, `gpio_slow_sel` stay at their existing all-zero tie (CMOS input, standard trip) — same as the other bidirectional pads.

Why it still carries risk: the whole per-pad config table rests on an item your tape-in notes flag — *the exact `gpio_dm / inp_dis` encoding was never verified against the real OpenFrame template; TODO in the wrapper header, confirm before GATE-3*. Four pads that switch direction at runtime make that verification mandatory. It is the same verification the QSPI and I²C pads need anyway, so this adds no new unknown — it raises the cost of an existing one.

Bridge safety is preserved: the low-side pads `io[27] AL`, `io[29] BL`, `io[31] CL` are untouched and remain `PC_OUT`. When `PINMUX[0]=0` (PWM role) `pad_oe` is forced high for all four muxed pads, so they behave exactly as today.

Two more pad-level checks for the reviewer: 1. **Idle level.** `PC_BIDIR` (`dm=110`) has **no weak pull** — the pull modes (`dm=010/011`) are input-only classes in this wrapper, and `gpio_dm` is a static tie, not switchable at runtime. So when the pad is input and the FC is not driving, the line floats. Bidirectional DShot idles *high*, normal DShot idles *low*. **Recommendation: require the external pull on the board** (bidirectional-DShot ESC wiring already puts a pull-up on the FC side) and document it in the datasheet; do not attempt a runtime-switched `gpio_dm`. 2. **Reset.** `busy_q` resets to 0 (`rtl/dgrid_dshot.sv:74`) and `dshot_o` to 0, so the TX engine does not drive

out of reset. `pad_oe` must follow: input during `rst_n_i` low, so a resetting DG32 never drives a line the FC is driving. Note this **inverts the current reset behaviour** of these four pads (today they drive 0 out of reset as `PC_OUT`) — the board must not rely on a hard low from AH/BH/CH during reset. `AL/BL/CL` are unchanged.

8. Timing, cost and verification

Edge precision at 50 MHz (20 ns/cycle): the decision margin is `T1H - T0H`. DShot300: 63 cycles (± 630 ns) — trivial. DShot600: 31 cycles (± 310 ns) — comfortable. **DShot1200: 15 cycles (± 150 ns)** — works on paper, but the synchroniser's ± 1 -cycle sampling jitter plus FC-side jitter eats a third of it. **Recommend DShot600 as the supported maximum; DShot1200 best-effort.**

Area estimate: per channel — 12-bit width counter, 16-bit RX shifter, 4-bit CRC, RX FSM (~ 70 flops); 21-bit reply shifter, 12-bit period counter, GCR/transition encoder (~ 50 flops); status/flags (~ 8). $\times 4 \approx \sim 520$ flops plus shared registers (~ 100) and 8 synchroniser flops. Roughly **1,500–2,500 std cells** — about 3–5% of the lite die's 50,550. Consistent with Ayaz's "cheap": the TX engine closes at 172.9 MHz against 50 MHz, and RX has no deeper logic.

Verification plan (before integration, all standalone): 1. `tb_dshot_rx` — loopback: existing TX engine drives RX through a modelled pad at all four speeds; every throttle 0–2047 $\times$ telem 0/1; assert `RXFRAME == transmitted` and `VALID`. Also inject each single-bit corruption $\rightarrow$ assert `CRC_ERR`, never `VALID`. 2. Glitch/desync: inject a 40 ns glitch mid-frame; assert timeout resync and that the *next* frame decodes clean. 3. Overrun: two frames back-to-back without a read $\rightarrow$ `OVERRUN`. 4. `tb_dshot_tel` — reply stream captured by a **reference FC decoder** implementing $G = s \wedge (s \gg 1) + \text{GCR table} + \text{CRC check}$, written independently from the encoder (not shared code). All 4096 period values. 5. Turnaround: assert `pad_oe` rises exactly `TURNAROUND` cycles after the last command edge and falls after bit 21; assert abort-on-incoming-edge releases OE within 1 cycle. 6. Bidirectional end-to-end: FC model sends inverted frame with inverted CRC; RX decodes; reply goes out inverted at 5/4 rate; FC model validates. 7. Integrated: re-run `fwtest` T-suite (T01–T17) bit-identical, plus a new **T18 dshot_rx loopback** on the FPGA using a jumper between two DShot pads.

Lockstep: RX is a normal bus slave. Its registers are read by MAIN and mirrored to CHECKER like every other peripheral; no store timing depends on the asynchronous input, because decoding is in hardware. This is the property firmware bit-banging cannot provide.

9. SRAM: 28 KB vs 32 KB — separating the two questions

Ayaz: "28 KB bare minimum, 32 KB with safety margin; the reusable buffer saves only 2 KB." That is a **firmware footprint** statement, and it is the right way to size memory.

The 28 KB proposal was a **floorplan lever**: 16 $\rightarrow$ 14 macros lets 2dom's 17 macros fit 3 columns $\times$ 6 rows instead of 3 $\times$ 7, deleting an entire macro row and growing the full-width logic strip from 1.75 mm² to 3.40 mm² — that is what makes 2dom fit the 2,900 μm wrapper slot at all. It was never a claim that firmware needs less memory.

Both can be true. The question is whether 2dom can fit at 2,900 µm **with** 32 KB, and the codex branch is already answering it empirically: 2dom/13 is a keep-32 KB spread-floorplan test, 2dom/12 a 24 KB variant. **If 13 routes and closes, keep 32 KB and drop the 28 KB idea. If only the reduced-SRAM variants fit, then the firmware team gets 28 KB and the 2 KB reusable buffer stops being optional.** Decide from those two runs, not from either side's estimate. Note the lite die is unaffected either way — it stays 32 KB.

10. What to implement — the list

RTL 1. rtl/dgrid_dshot_rx.sv — per-channel RX decoder (§4.1), 4 instances, parameterised on TW=12. 2. rtl/dgrid_dshot_tel.sv — per-channel GCR/transition reply engine (§4.2). 3. Extend dgrid_dshot.sv: registers 0x24–0x5C (§5), pad_i[3:0] input, pad_oe[3:0] output, irq_o, direction FSM (§4.3). Existing 0x00–0x20 behaviour and the TX engine unchanged; tb_dshot must still pass bit-identical. 4. dg32_lite_top.sv: route pad_i/pad_oe for the four muxed pads; OE forced high in PWM role; new IRQ source into dgrid_irq.

Wrapper / pad ring 5. openframe_project_wrapper.v: io[26/28/30/32] PC_OUT → PC_BIDIR in pad_class(); oeb_v for those four from ~pad_oe via u_glue; gpio_in for those four to the top; OE deasserted in reset. Board-side pull documented (no on-die pull in this class). 6. **Verify the OpenFrame per-pad control encoding against the real template** — already a GATE-3 TODO; now blocking.

Firmware / docs 7. dg32.h: register defines; fwtest T18 loopback test. 8. Datasheet: GPIO 14 → **3**; DShot bidirectional "not supported" → "supported, DShot150–600, best-effort 1200"; document the pad-direction behaviour. 9. DG_lite_analysis.md item 9: reopen.

Verification — §8 items 1–7.

Not in scope of this change: a new bus slot (kept at 0xC), timer input capture (not needed once RX is hardware), any change to the low-side PWM pads.

11. Open questions for Ayaz before implementation starts

1. Roles per pad — is DG32 ever *both* FC (TX) and ESC (RX) on different channels simultaneously? The spec allows it (per-channel direction), but if not required, a global role bit simplifies the FSM.
2. Reply on every frame (TELEM_ALWAYS) vs only on telem-request — which FC firmware are you targeting first? Betaflight bidirectional expects every frame.
3. Idle level: the bidirectional pad class has no on-die pull (§7). Is a board-side pull-up acceptable, or do you need the die to define the idle level? (The latter means a runtime-switched gpio_dm, which I recommend against.)
4. Reset behaviour change: AH/BH/CH/TRIG will float (input) during reset instead of driving 0. Is anything on your board relying on the hard low?
5. Priority vs tape-in: this touches the wrapper's unverified pad config. It is **not** for the 16 Sep die. Confirm it targets the following shuttle (CI2612) alongside 2dom.